

2-D Feed Backward Predictive Coding

Course Information

Course: DSAI 325 – Introduction to Information Theory

Assignment: Assignment 5 – JAVA Implementation of 2-D Feed Backward Predictive Coding

Semester: Spring 2026

University: University of Science and Technology

School: School of Computational Sciences and Artificial Intelligence

Table of Contents

1. Introduction
2. Assignment Objectives
3. Theory Background
4. Predictive Coding Overview
5. System Design and Workflow
6. Implementation Details
7. Predictor Types
8. Quantization Process
9. Compression Ratio and Quality Metrics
10. Experimental Setup
11. Results and Performance Analysis
12. Reconstruction Quality Analysis
13. Output Images Section

- 14. Discussion and Observations
 - 15. Conclusion
 - 16. Appendix – Important Code Sections
-

1. Introduction

Image compression is one of the most important applications in information theory and multimedia systems. Predictive coding techniques reduce redundancy between neighboring pixels by predicting pixel values using previously reconstructed pixels. Instead of storing the original pixel values directly, only the prediction errors (residuals) are encoded and stored.

In this assignment, a complete JAVA implementation of **2-D feed-backward predictive coding** was developed and tested using different predictor types and different quantization levels. The implementation performs both encoding and decoding, reconstructs the image, and evaluates the performance using numerical quality metrics such as

- Mean Squared Error (MSE)
- Peak Signal-to-Noise Ratio (PSNR)
- Compression Ratio (CR)

The assignment also analyzes the effect of:

- Predictor type
- Quantization level

on both image quality and compression performance.

2. Assignment Objectives

The main objectives of this assignment are:

- Implement a complete 2-D Feed Backward Predictive Coding system in JAVA.
 - Apply multiple predictor models:
 - Order-1 Predictor
 - Order-2 Predictor
 - Adaptive 2-D Predictor
 - Implement uniform scalar quantization.
 - Reconstruct the compressed image using decoding.
 - Calculate image quality metrics:
 - MSE
 - PSNR
 - Compression Ratio
 - Compare the performance of different predictors and quantization levels.
 - Analyze the trade-off between compression efficiency and reconstruction quality.
-

3. Theory Background

Predictive coding is a lossy compression technique based on the assumption that neighboring pixels in images are highly correlated.

Instead of encoding the original image pixels directly, the encoder predicts the current pixel value using previously reconstructed neighboring pixels.

$$e(i, j) = x(i, j) - \hat{x}(i, j)$$

Where:

- $x(i, j)$ = Original pixel value
- $\hat{x}(i, j)$ = Predicted pixel value
- $e(i, j)$ = Prediction error (residual)

The residual values are then quantized and stored. During decoding, the image is reconstructed using the same predictor and the quantized residuals.

4. Predictive Coding Overview

The implemented predictive coding system consists of two main stages:

4.1 Encoder

The encoder performs:

1. Image preprocessing
2. Pixel prediction
3. Residual calculation
4. Quantization
5. Storage of quantized residuals

4.2 Decoder

The decoder performs:

1. Dequantization
 2. Pixel reconstruction
 3. Rebuilding the image
 4. Saving reconstructed image
-

5. System Design and Workflow

The complete workflow of the system is:

1. Read input image
 2. Convert image to grayscale
 3. Store image pixels in a 2D array
 4. Predict each pixel value
 5. Calculate prediction error
 6. Quantize residuals
 7. Store compressed representation
 8. Decode residuals
 9. Reconstruct image
 10. Compute quality metrics
 11. Save reconstructed image
-

6. Implementation Details

The project was implemented using JAVA.

Main Files

1. PredictiveCoding.java

Contains:

- Encoder implementation
- Decoder implementation
- Prediction functions
- Quantization functions
- Quality metric calculations

2. TestPredictiveCoding.java

Used for:

- Running all test cases
 - Comparing predictors
 - Comparing quantization levels
 - Printing results summary
-

7. Predictor Types

7.1 Order-1 Predictor

The Order-1 predictor uses the left neighboring pixel:

$$\hat{x}(i, j) = x(i, j - 1)$$

Characteristics

- Simple implementation
 - Fast computation
 - Moderate prediction accuracy
-

7.2 Order-2 Predictor

The Order-2 predictor uses:

- Left pixel
- Upper pixel
- Upper-left pixel

Formula:

$$\hat{x}(i, j) = x(i, j - 1) + x(i - 1, j) - x(i - 1, j - 1)$$

Characteristics

- Better prediction accuracy
 - Uses more neighboring information
 - More computational complexity
-

7.3 Adaptive 2-D Predictor

The adaptive predictor dynamically selects prediction values based on local image characteristics.

Characteristics

- Higher adaptability
 - Better reconstruction quality
 - Improved handling of image edges and textures
-

8. Quantization Process

Uniform scalar quantization was used to reduce the number of residual values.

Quantization levels tested:

- 8 levels
- 16 levels
- 32 levels

Quantization Effect

- Lower levels:
 - Higher compression
 - Lower image quality
 - Higher levels:
 - Lower compression
 - Better image quality
-

9. Compression Ratio and Quality Metrics

9.1 Mean Squared Error (MSE)

MSE measures the reconstruction error between the original and reconstructed image.

$$MSE = \frac{1}{MN} \sum_{i=1}^M \sum_{j=1}^N (x(i, j) - \hat{x}(i, j))^2$$

Lower MSE indicates better reconstruction quality.

9.2 Peak Signal-to-Noise Ratio (PSNR)

PSNR evaluates reconstructed image quality.

$$PSNR = 10 \log_{10} \left(\frac{255^2}{MSE} \right)$$

Higher PSNR values indicate better image quality.

9.3 Compression Ratio (CR)

Compression ratio measures compression efficiency.

$$CR = \frac{\text{Original Size}}{\text{Compressed Size}}$$

A higher compression ratio means better compression performance.

10. Experimental Setup

Test Image Information

Property	Value
Test Image	test_image.png
Image Type	Gray-scale
Total Pixels	65536
Original Size	65536 Bytes
Pixel Depth	8 bits/pixel

11. Results and Performance Analysis

Results Summary Table

Predictor	Q-Levels	MSE	PSNR (dB)	Compression Ratio
Order-1	8	82.7023	28.9556	1.6000
Order-1	16	18.3477	35.4950	1.3333
Order-1	32	4.0725	42.0322	1.1429

Order-2	8	82.7023	28.9556	1.6000
Order-2	16	18.2693	35.5136	1.3333
Order-2	32	4.0725	42.0322	1.1429
Adaptive 2-D	8	82.7023	28.9556	1.6000
Adaptive 2-D	16	18.1517	35.5416	1.3333
Adaptive 2-D	32	4.0725	42.0322	1.1429

12. Reconstruction Quality Analysis

Analysis by Quantization Level

Q = 8 Levels

- Highest compression ratio
- Lowest image quality
- Large reconstruction error
- All predictors produced similar performance

Q = 16 Levels

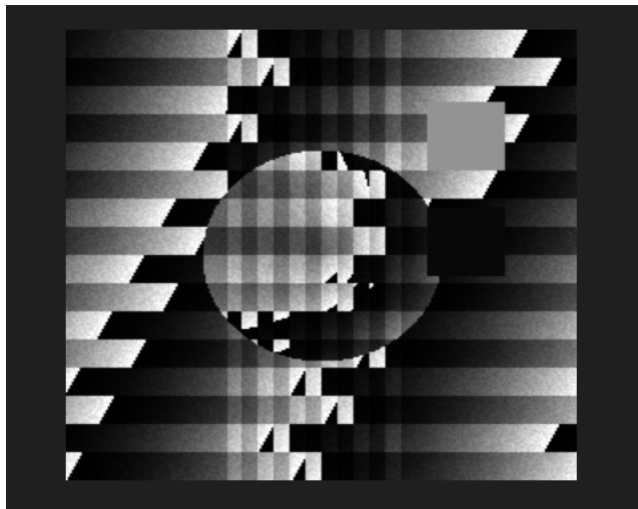
- Balanced compression and quality
- Adaptive predictor achieved the lowest MSE
- Better visual reconstruction

Q = 32 Levels

- Best image quality
 - Lowest reconstruction error
 - Lowest compression ratio
 - Very high PSNR values
-

13. Output Images Section

Original Image

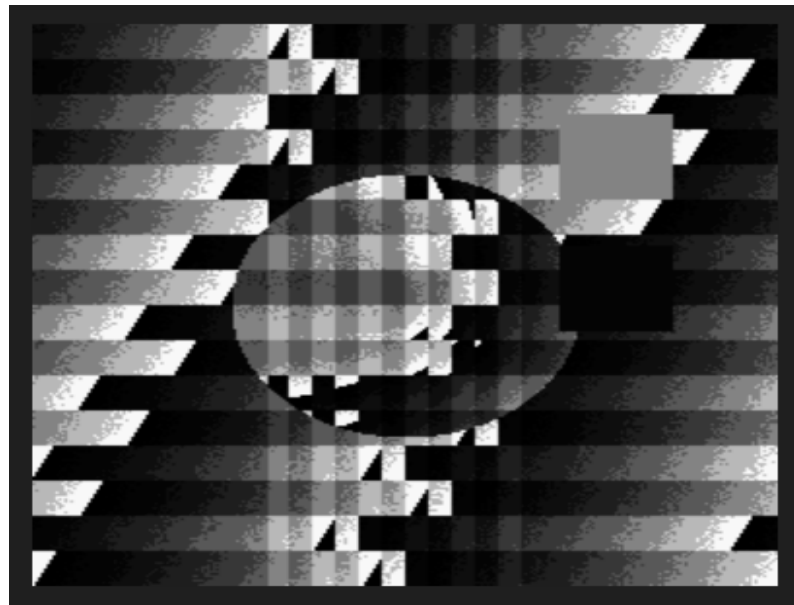


- Original grayscale image
- Input image before compression

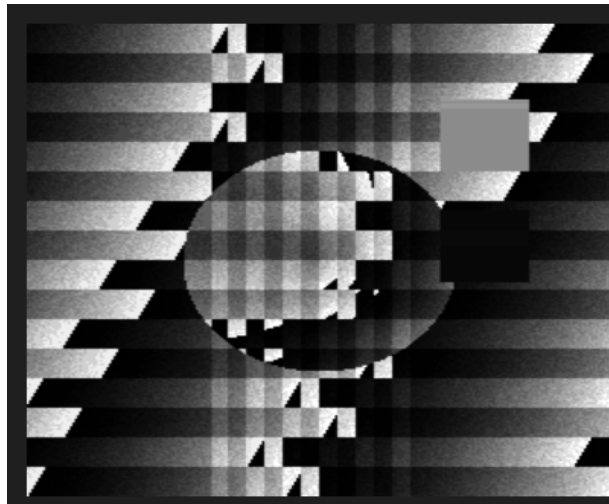
Reconstructed Images

Order-1 Predictor

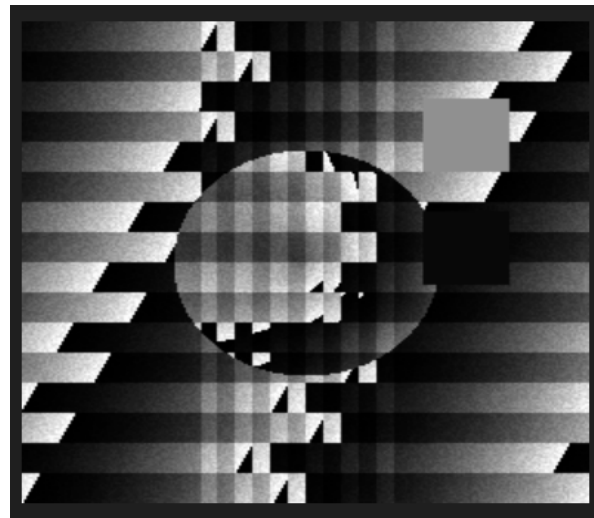
$Q = 8$



$Q = 16$

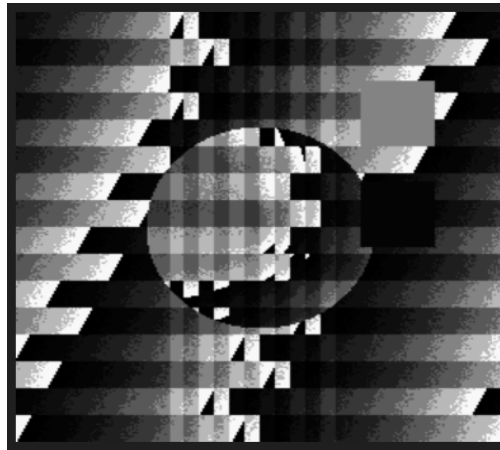


$Q = 32$

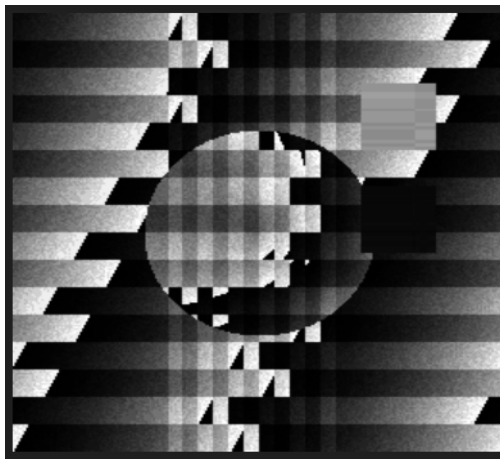


Order-2 Predictor

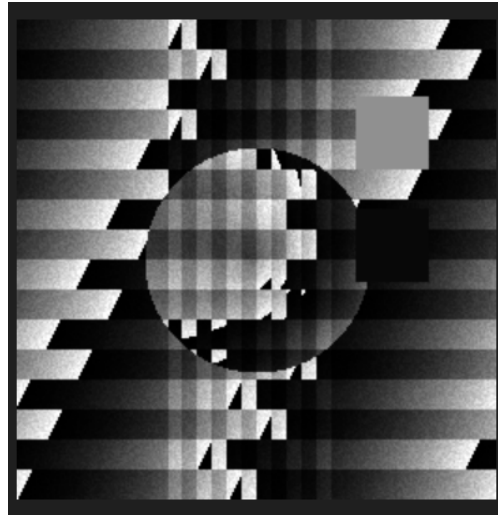
$Q = 8$



$Q = 16$

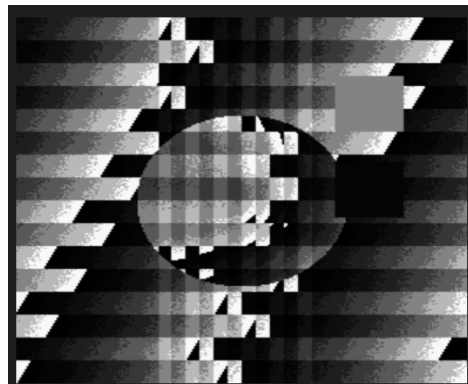


$Q = 32$

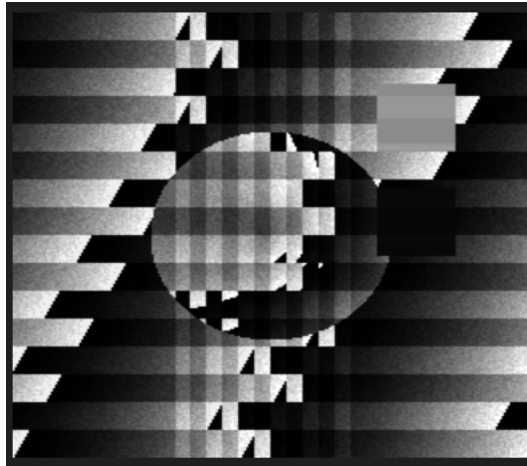


Adaptive 2-D Predictor

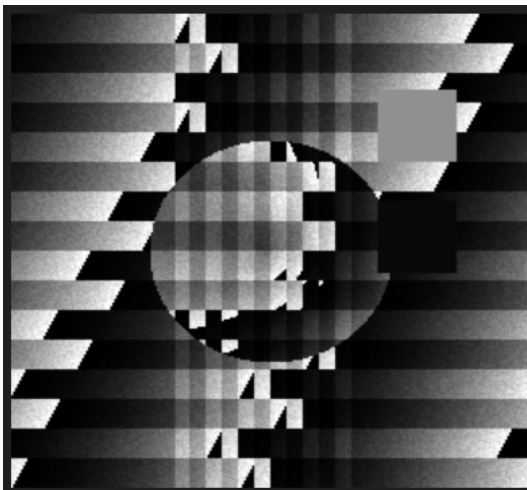
$Q = 8$



$Q = 16$



$Q = 32$



14. Discussion and Observations

Predictor Performance

Order-1 Predictor

- Simple and efficient
- Performs well in smooth image regions
- Lower computational complexity

Order-2 Predictor

- Provides improved prediction accuracy
- Better handling of edges
- Slight improvement over Order-1

Adaptive 2-D Predictor

- Best overall reconstruction quality
- Dynamically adapts to image characteristics
- Produces the lowest MSE at Q=16

Quantization Impact

Low Quantization Levels

Advantages:

- Higher compression ratio
- Reduced storage requirements

Disadvantages:

- Larger prediction error
- Lower reconstructed image quality

High Quantization Levels

Advantages:

- Better visual quality
- Lower MSE
- Higher PSNR

Disadvantages:

- Lower compression efficiency
-

Best Overall Result

The best reconstruction quality was achieved using:

- Predictor: Adaptive 2-D
- Quantization Level: 32
- MSE: 4.0725
- PSNR: 42.0322 dB

This configuration produced excellent reconstructed image quality with minimal visible distortion.

15. Conclusion

This assignment successfully implemented a complete 2-D Feed Backward Predictive Coding system using JAVA.

Different predictor models and quantization levels were tested and analyzed. The results demonstrated that:

- Predictive coding effectively reduces image redundancy.
- Increasing quantization levels improves reconstruction quality.
- Lower quantization levels increase compression efficiency.
- Adaptive prediction provides better reconstruction quality in most cases.
- There is always a trade-off between compression ratio and image quality.

The project provided practical understanding of:

- Predictive image compression
- Quantization
- Lossy image coding
- Reconstruction quality evaluation

Overall, the implementation achieved successful compression and reconstruction with meaningful analytical results.

16. Appendix

Main Functions Implemented

convertToGrayscale()

Converts RGB image into grayscale representation.

performPredictiveCoding()

Performs:

- Prediction
- Quantization
- Encoding
- Decoding

predictPixel()

Implements:

- Order-1 Predictor
- Order-2 Predictor
- Adaptive Predictor

quantizeError()

Performs uniform scalar quantization.

dequantizeError()

Restores approximate residual values.

calculateMSE()

Calculates reconstruction error.

calculateCompressionRatio()

Calculates compression efficiency.

Output Files

The following reconstructed images were generated:

- output/reconstructed_pred1_q8.png
- output/reconstructed_pred1_q16.png
- output/reconstructed_pred1_q32.png
- output/reconstructed_pred2_q8.png
- output/reconstructed_pred2_q16.png
- output/reconstructed_pred2_q32.png
- output/reconstructed_pred3_q8.png
- output/reconstructed_pred3_q16.png
- output/reconstructed_pred3_q32.png